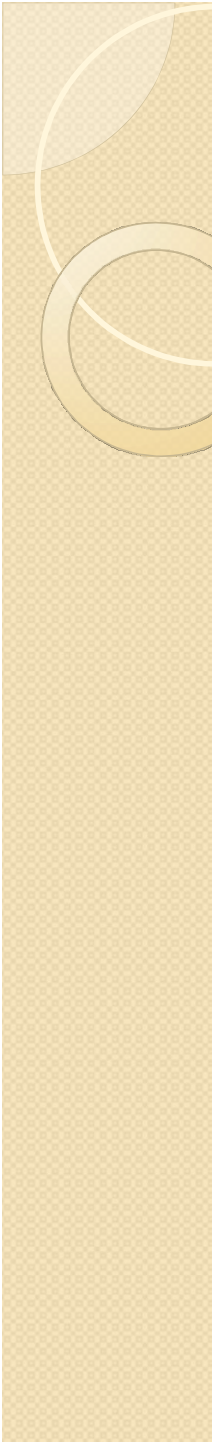


Module 2:- AI Foundation
Lecture 5:-

Deep Learning & Neural Networks

**Understanding the Technology Behind
Modern AI**

CNN • RNN • LSTM • Backpropagation •
Forward Pass • AI Applications



Slide 1 – Why Deep Learning?

Traditional Programming

Rules + Data



Output

Machine Learning

Data + Labels



Learn Rules

Deep Learning

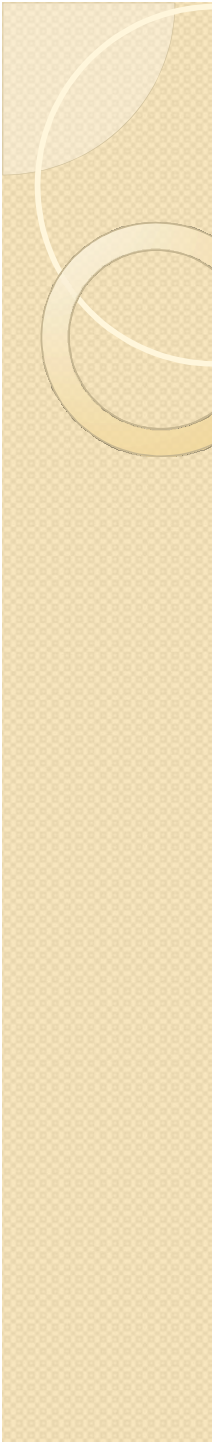
Raw Data



Automatically Learns Features



Prediction



Why Deep Learning?

Learns directly from large datasets

No manual feature engineering

Excels at images, speech, text, and video

Foundation of modern AI systems

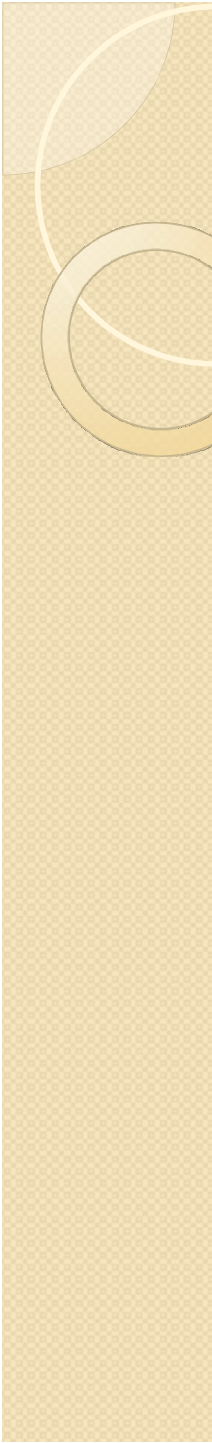
Examples:

ChatGPT

Google Lens

Face Unlock

Tesla Autopilot



Slide 2 – Evolution of AI

Artificial Intelligence



Machine Learning



Deep Learning



Transformers

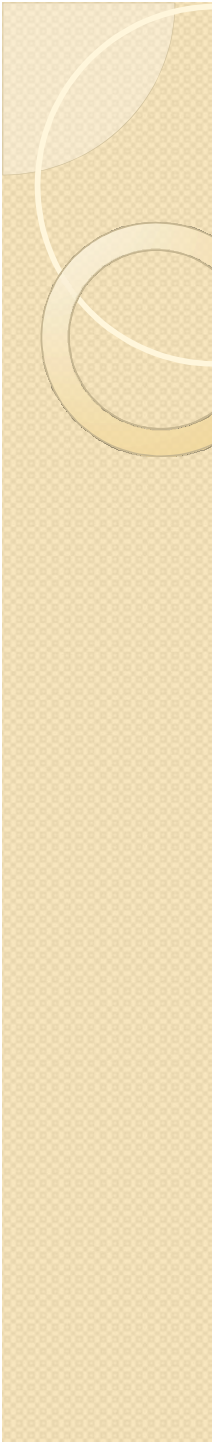


Large Language Models



ChatGPT

Every generation solved limitations of the previous one.



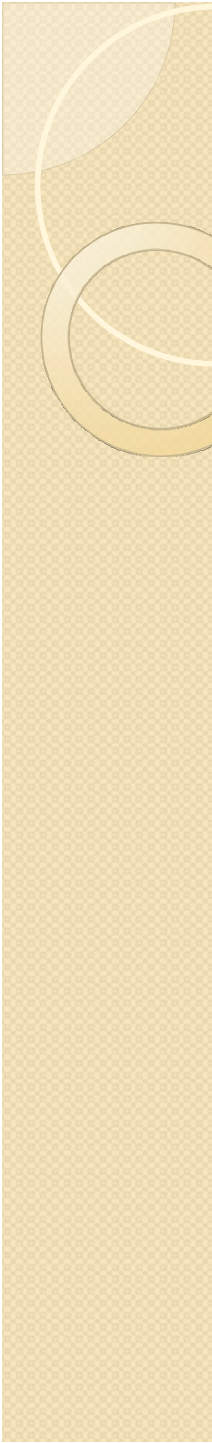
Slide 3 – What is Deep Learning?

Definition

Deep Learning is a subset of Machine Learning that uses Artificial Neural Networks with multiple hidden layers to automatically learn hierarchical patterns from data.

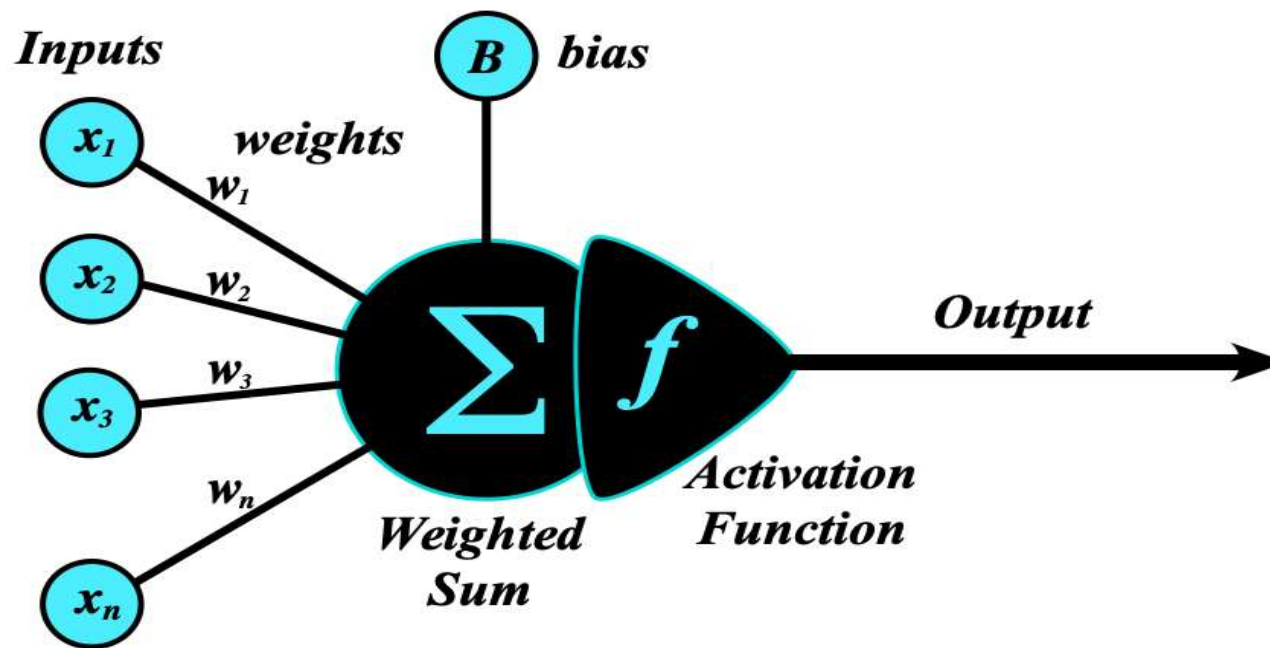
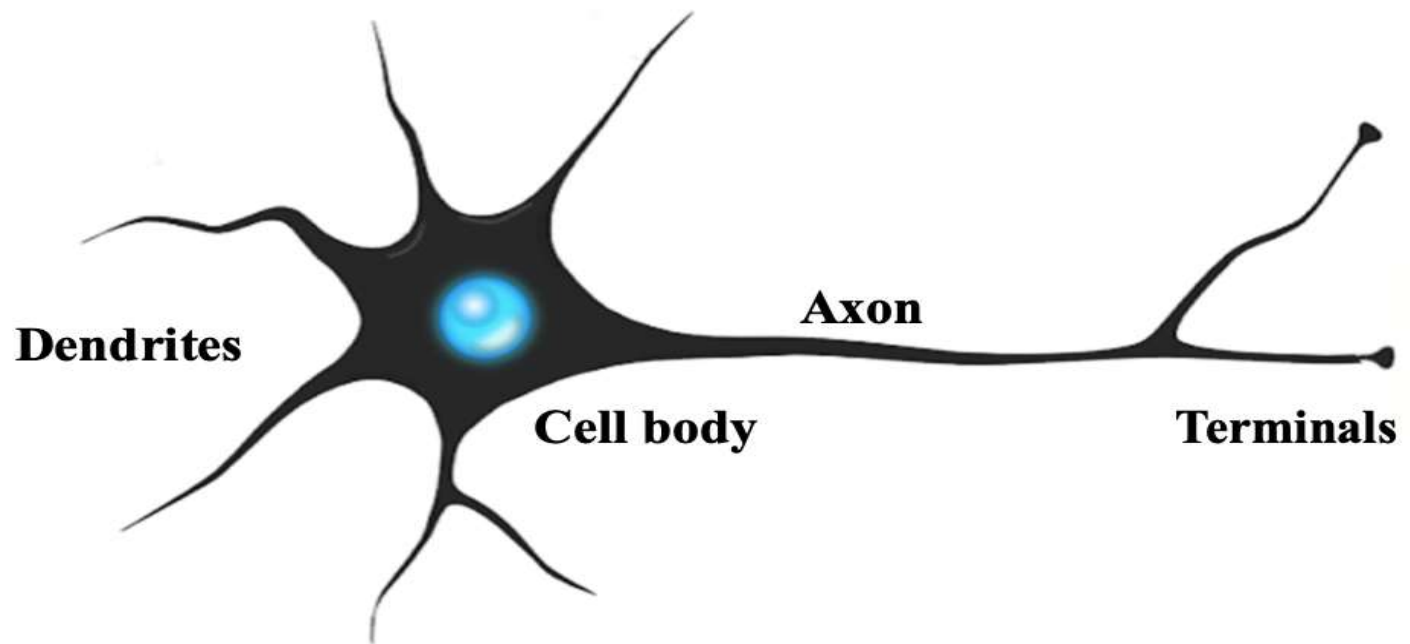
Characteristics

- Data-driven
- Multiple hidden layers
- Learns features automatically
- Requires GPUs
- Handles unstructured data



Slide 4 – Human Brain vs Artificial Neural Network

Human Brain	Neural Network
Neurons	Artificial Neurons
Synapses	Weights
Learning	Training
Memory	Parameters
Decision	Prediction



Slide 5 – Artificial Neuron

Every neuron receives inputs multiplies by weights adds bias
passes through activation produces output

Terms:-

Input

Weight

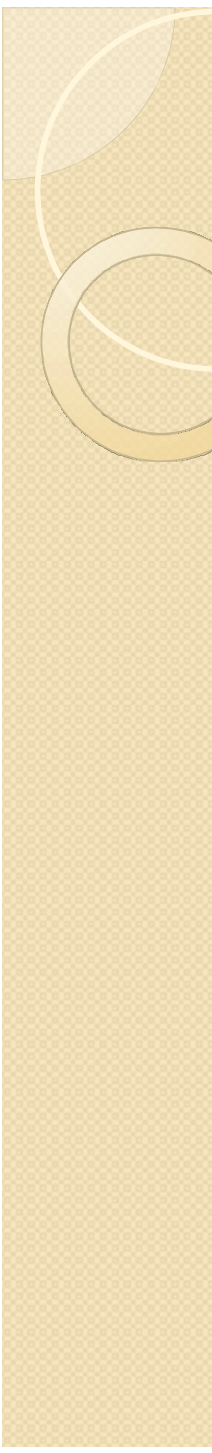
Bias

Activation

Output

$$\text{Output} = \text{Activation} \left(\sum (\text{Input} \times \text{Weight}) + \text{Bias} \right)$$

- 
- **Input (x):** The incoming data or signals. If you are training a model to predict house prices, the inputs might be numbers representing Square Footage or Number of Bedrooms.
 - **Weight (w):** The **importance or strength** assigned to an input. If Square Footage matters significantly more than the color of the front door, the model assigns a much larger weight value to square footage.
 - **Bias (b):** An extra value added to the sum. It acts like an offset, shifting the entire mathematical relationship up or down to ensure the model fits the data accurately, even if all inputs are zero.

- 
- **Activation Function (f):** A mathematical formula that decides whether the neuron should "fire" (and how strongly).

Common types include **ReLU, Sigmoid, and Tanh.**

- **Output (y):** The final signal calculated by the neuron. This value is either sent to the next layer of neurons or serves as the network's final prediction.

Numerical Example

Imagine a neuron deciding if a house price will be high based on two inputs:

Inputs (x):

$x_1 = 3$ (Rooms)

$x_2 = 2$ (Bathrooms)

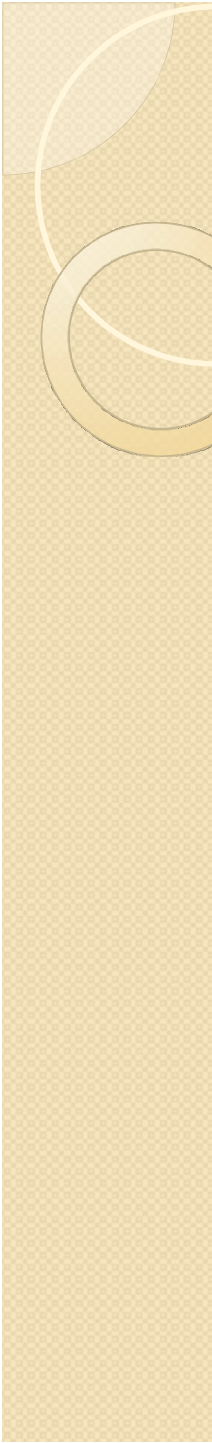
Weights (w):

$w_1 = 0.5$ (Importance of rooms)

$w_2 = 0.8$ (Importance of bathrooms)

Bias (b): -1.0 (Offsets the calculation)

Activation Function: ReLU (If the total is negative, output is 0. Otherwise, keep the number).



Step-by-Step Calculation:

Multiply & Sum: $(3 \times 0.5) + (2 \times 0.8) = 1.5 + 1.6 = 3.1$

Add Bias: $3.1 + (-1.0) = 2.1$

Activation (ReLU): Since 2.1 is positive, it stays **2.1**.

Output: 2.1

python

```
def relu(x):  
    return max(0, x)
```

1. Define inputs, weights, and bias

```
inputs = [3.0, 2.0]  
weights = [0.5, 0.8]  
bias = -1.0
```

2. Multiply by weights and sum them up

```
weighted_sum = sum(i * w for i, w in zip(inputs, weights))
```

3. Add bias

```
total = weighted_sum + bias
```

4. Pass through activation function to produce output

```
output = relu(total)
```

```
print(f"Neuron Output: {output}") # Output: 2.1
```

Slide 6 – Neural Network Architecture

Input Layer



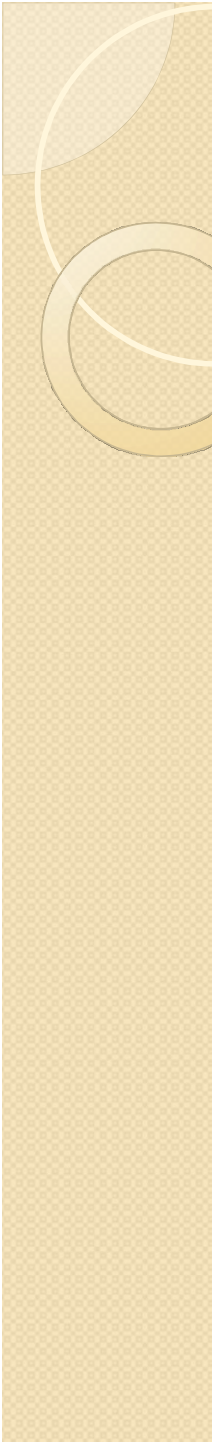
Hidden Layer



Hidden Layer



Output Layer



Input Layer

Receives raw data

Example

Pixels

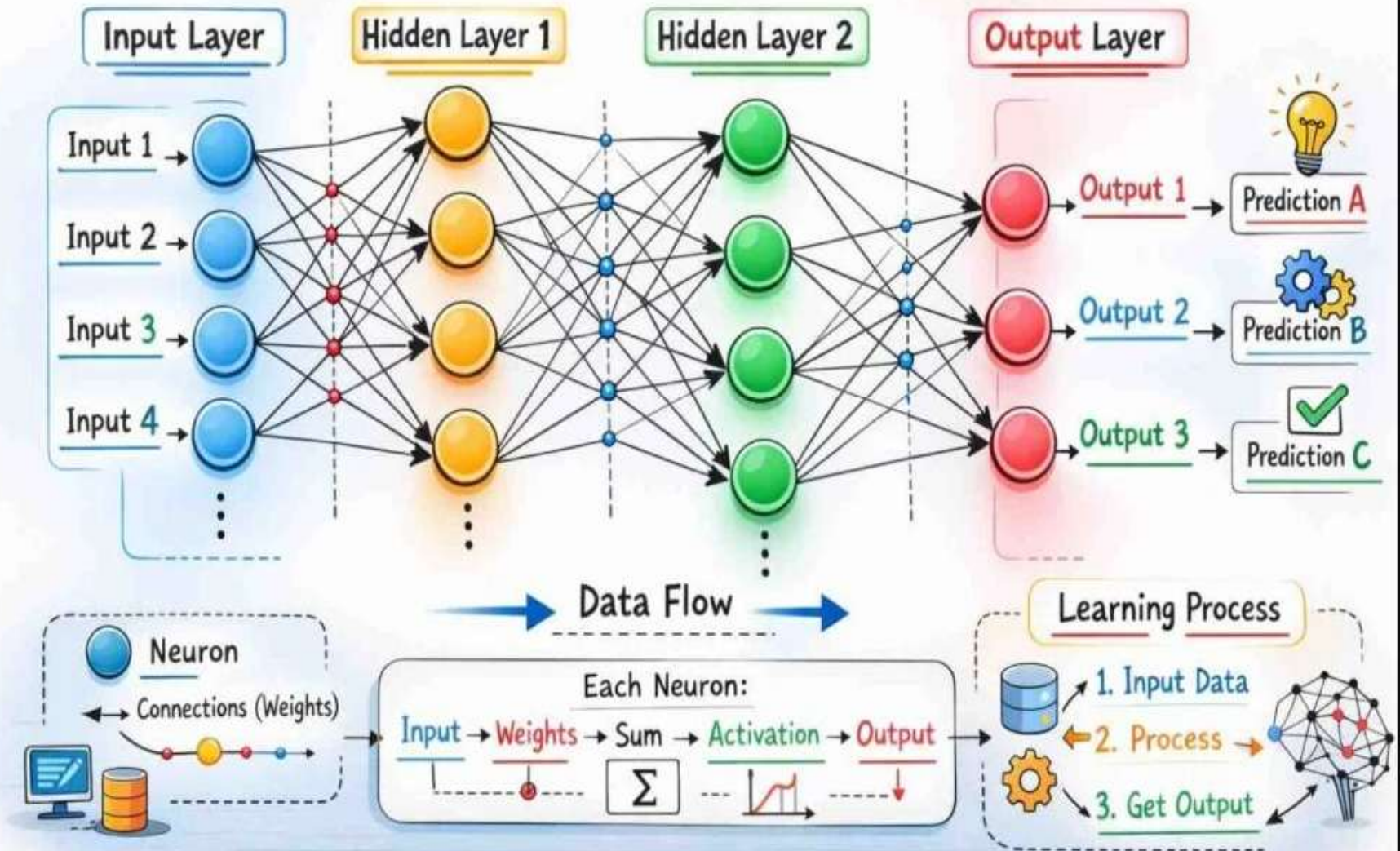
Hidden Layer

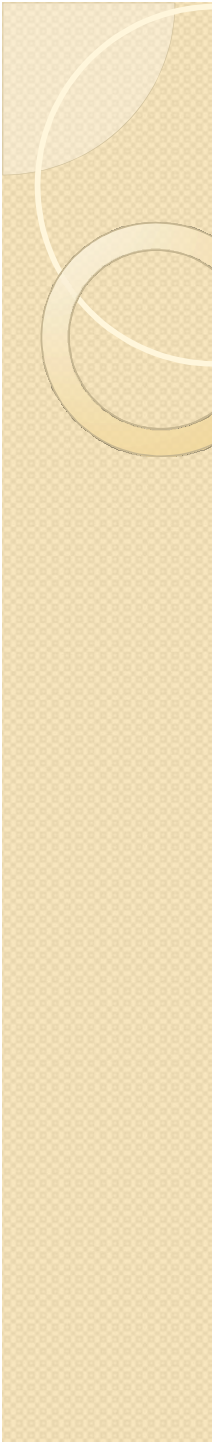
Extracts patterns

Output Layer

Final prediction

ARTIFICIAL NEURAL NETWORK





Slide 7 – Hidden Layers

Layer 1

Learns edges



Layer 2

Learns textures



Layer 3

Learns shapes



Layer 4

Learns complete object

Example

Image



Edges



Eyes

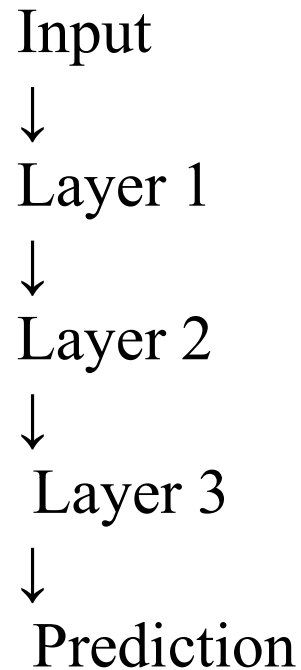


Face



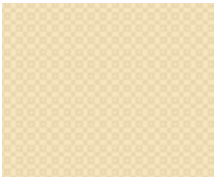
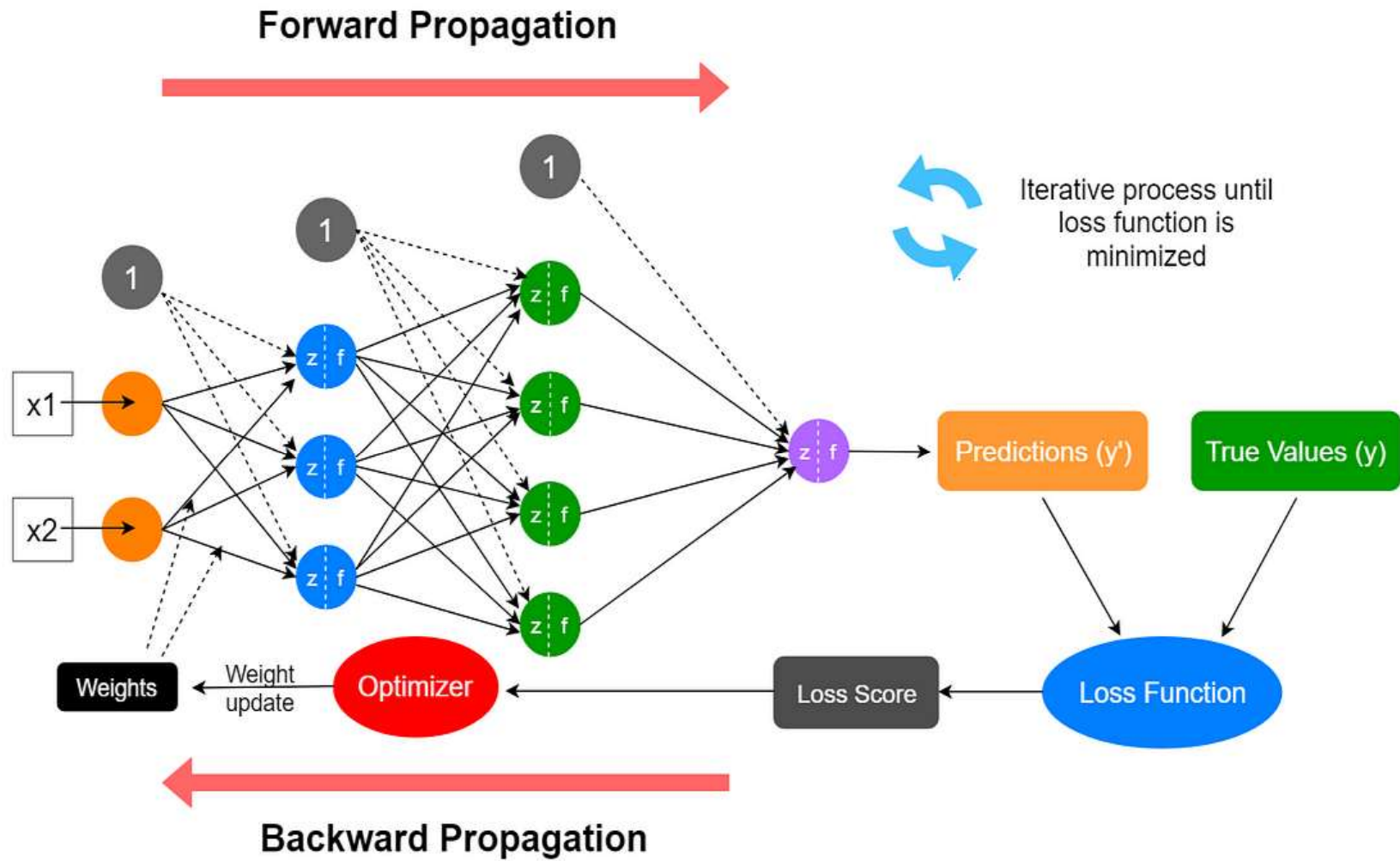
Person

Slide 8 – Forward Propagation



Definition

Forward propagation is the process where data flows from input to output to generate a prediction.



Slide 9 – Loss Function

Prediction



Compare with Actual



Calculate Error

Example

Predicted:- Dog

Actual:- Cat

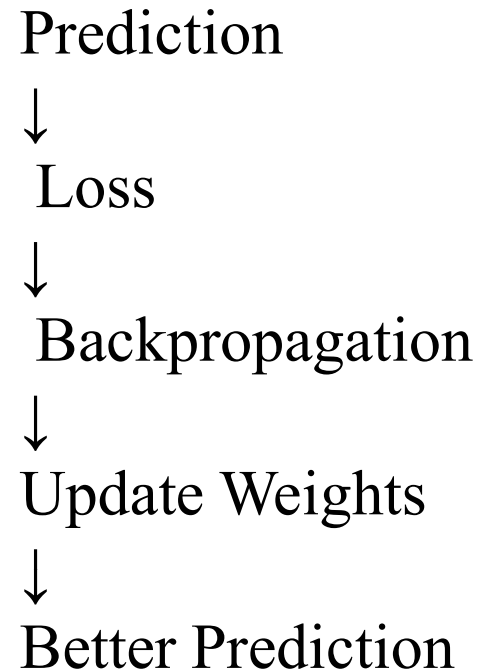


High Error

Loss measures how wrong the prediction is using methods like:-

- **Mean Squared Error**
- **Cross Entropy**

Backpropagation



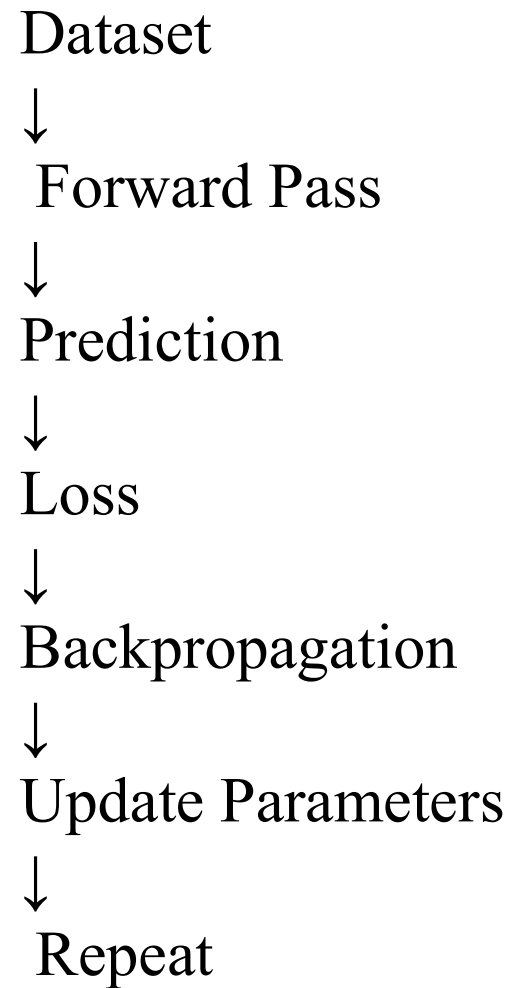
Definition

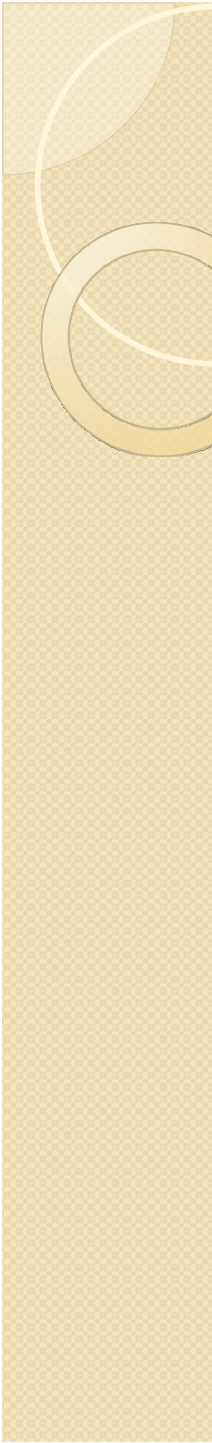
Backpropagation updates network weights to minimize prediction error.

Example:-

Learning = Repeated correction

Slide 11 – Training Process

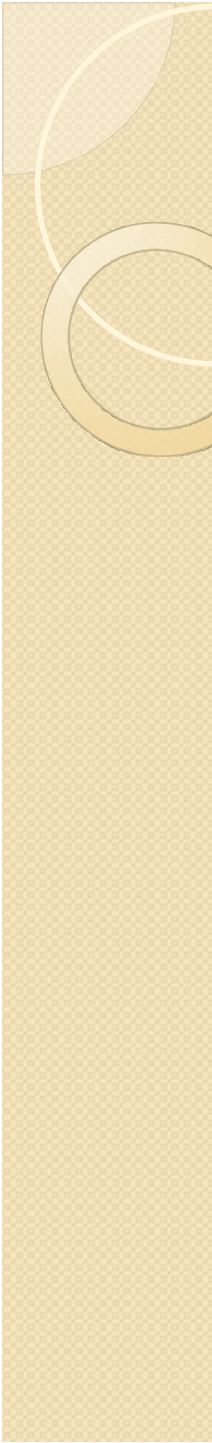




Batch: A chunk (or group) of data fed into the network simultaneously. Example: If you have 10,000 images and train using subsets of 100 images at a time, the "batch size" is 100.

Iteration: The number of times a "parameter update" (the loop described above) is performed. Example: If processing 10,000 images in batches of 100, you must repeat the loop 100 times (= 100 iterations) to process all the data.

Epoch: A unit representing one complete pass through the entire training dataset. Example: Once all 10,000 images have been used (taking 100 iterations), "1 epoch" is complete. To make the model smarter, this process is typically repeated over dozens or hundreds of epochs.



Imagine you are preparing for a massive final exam by studying a **1,000-page textbook**.

Epoch: Reading the **entire textbook** from cover to cover exactly one time.

Batch: The **number of pages** you choose to read at a single time before taking a short break (e.g., 100 pages per study session).

Iteration: The **actual study session** itself. Every time you finish a batch of 100 pages and pause to think about what you learned, you have completed one iteration.

Slide 12 – Activation Functions

Purpose

Introduce non-linearity. Activation functions introduce non-linearity. They bend, curve, and twist the data graph. This allows the AI to learn complex, wavy, and intricate patterns.

Types

Sigmoid

ReLU

Tanh

Softmax

ReLU is most common today.

No mathematics.

Slide 13 – CNN

Convolutional Neural Network

Specialized for images.

Architecture

Image (the raw data)



Convolution (Scanning the image to look for specific patterns.)



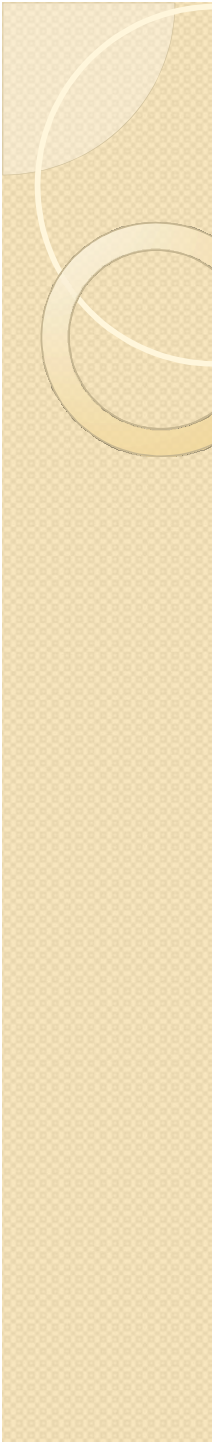
Pooling (Reducing the size of the data while keeping the important parts.)



Fully Connected Layer (The final reasoning step where all clues are combined.)



Prediction(The output decision.)



Simple Everyday Analogies for Each Step

Image

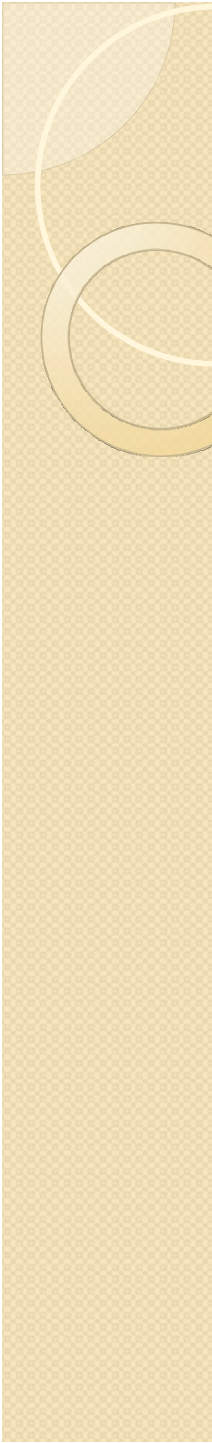
Analogy: A large crime scene photo handed to a detective.

Filters

Analogy: A small, magical magnifying glass slid across the photo. It glows only when it passes over specific shapes like sharp noses or round eyes.

Feature Maps

Analogy: A notepad of clues created by the magnifying glass, marking down exactly where those special shapes and lines were found.



Pooling

Analogy: Squinting your eyes at the notepad. It blurs out the tiny, useless details and leaves only the biggest, most important clues on a smaller page.

Classification

Analogy: The final detective report. The brain puts all the remaining clues together and makes a final guess: "This is a picture of Cat Burglar A!"

Slide 14 – CNN Working

Image



Filters



Feature Maps



Pooling



Classification

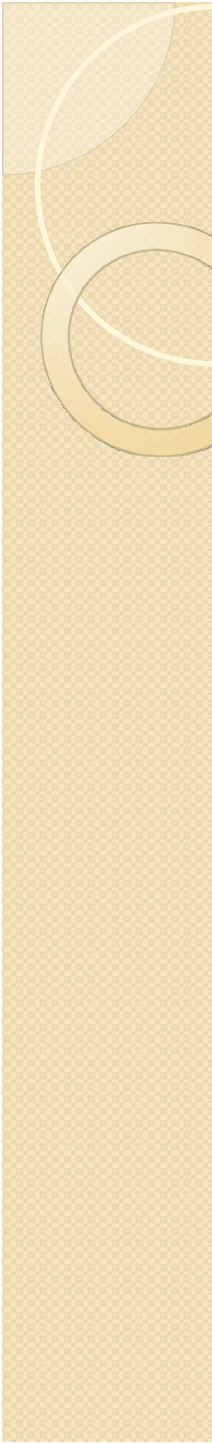
CNN automatically detects

Edges

Corners

Shapes

Objects



Slide 15 – Pooling Layer

Purpose

Reduce image size while preserving important information.

Types

Max Pooling

Average Pooling

Benefits

Faster computation

Less memory

Better generalization

Slide 16 – RNN Recurrent Neural Network

Designed for sequential data.

Word₁



Word₂



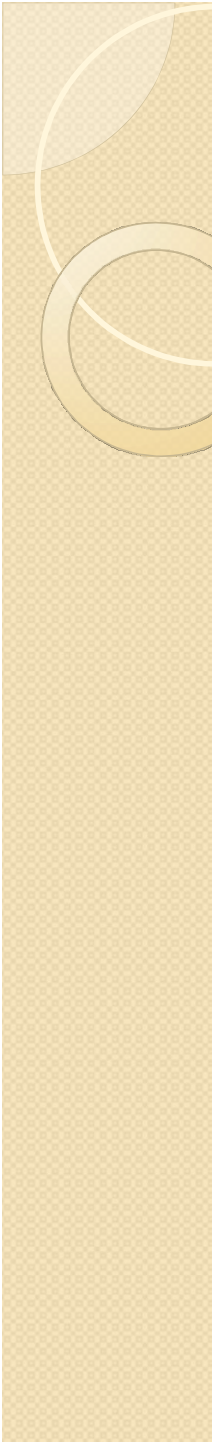
Word₃



Prediction

Applications

- Language Translation
- Speech Recognition
- Text Prediction



Slide 17 – Limitation of RNN

Problem

Long-term dependencies.

Example

The boy who won the competition after several rounds
received his prize.

The network may forget "The boy"

Slide 18 – LSTM

Long Short-Term Memory

Adds memory cells.

Input



Forget Gate



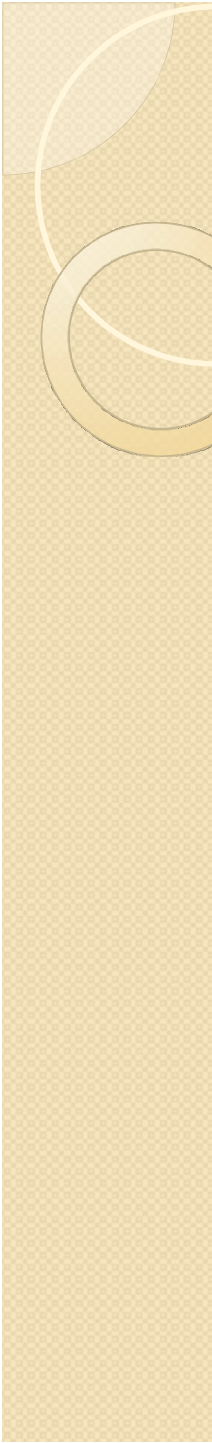
Input Gate



Output Gate



Prediction

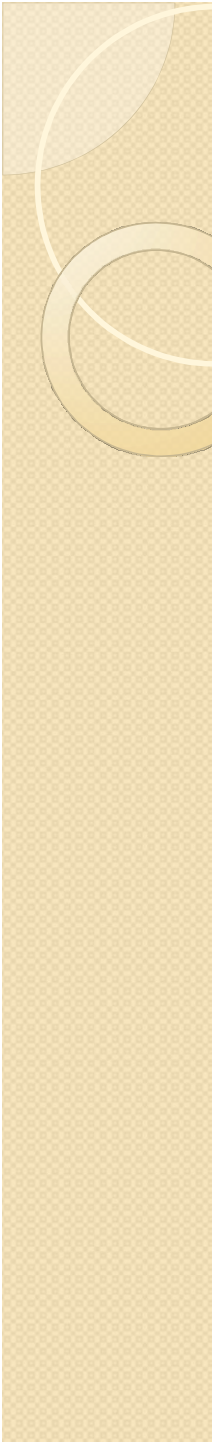


Purpose

Remember important information over long sequences.

Applications

- Translation
- Voice Assistant
- Time Series
- Speech Recognition



CNN	RNN	LSTM
Images	Sequential Data	Long Sequential Data
Parallel	Sequential	Sequential
Face Recognition	Translation	Chatbot Memory
Computer Vision	Speech	Weather Prediction

Slide 22 – Industry Deep Learning Stack

Python



NumPy



Pandas



TensorFlow / PyTorch



CUDA (parallel computing model, NVIDIA)



GPU



Model Training



Deployment



API



Application

Slide 23 – Future

ANN



CNN



RNN



LSTM



Transformers



LLMs



AI Agents

↓ AGI?

Transition

"Today we understood **how Deep Learning works**. In the next lecture, we'll explore **Transformers**, the breakthrough architecture that powers ChatGPT, Gemini, Claude, and modern Large Language Models."

- **Slide 24 – Key Takeaways**

- Deep Learning is a subset of Machine Learning.
- Neural Networks are inspired by the human brain.
- Hidden layers learn increasingly complex features.
- Forward Propagation makes predictions.
- Loss measures error.
- Backpropagation improves the model.
- CNN excels at images.
- RNN processes sequences.
- LSTM handles long-term dependencies.
- Modern LLMs are based on **Transformers**, not RNNs or LSTMs.